DATS 598: Data Science Capstone

Project Outline

A Statistical Truth Detection System

Eric Crisp
University of Pennsylvania
ecrisp@upenn.edu

Tuesday 7th October, 2025

Contents

Video Demo Link: click here.

Problem Definition and Background

This project is about taking input and determining if the statements within are true or false. Applications of this work include a variety of situations such as real-time news verification during debates, reports, or other media outlets, in researching the methods for retrieving evidence and verifying truth via RAG, embeddings, or other techniques, and finally the influence of truth in LLMs including response generation with and without truth verification.

The general result at the end of the day is a backend Python script that processes input and determines the truthfulness of the statements (and inferences) within. The backend will be wrapped around a simple web app for demonstration purposes, however, the main focus is the backend processing and truth verification. The application will be reflect a simple text prompt and will result in an analyzed output depending on the input and user configuration.

Literature Review

Much of the literature relates to the process of handling sentence detection and evidence retrieval. White papers, or survey papers, on the field of automated fact-checking consist of generally three categories: preprocessing, ranking and evidence retrieval, and multi-classification, or a degree of truthfulness. Preprocessing typically consists of two methods, a subject-predicate-object (SPO) triplet extraction, or a sentence classification and embedding approach. Ranking and evidence retrieval typically consists of a similarity-based approach, a more advanced RAG technique, or perhaps more traditionally knowledge graph path traversal ranking. Document retrieval is one of the more challenging parts because of the potentially ambiguous nature of the input. NLP techniques require being able to consistently decipher named entities and context understanding just to correctly identify the implied, or explicitly stated claim.

The most useful outcome from the literature review is two fold: understanding the processes and challenges involved with preprocessing as that appears to be the most consistent hurdle, and consistent reliable databases to pull from rather than requiring a web crawler. Datasets such as FEVER, Wikidata, Claimbuster, ClueWeb, Factify are all open source, available databases that can be incorporated into the project for evidence retrieval and truth verification. The creativity comes with how to use the results of retrieval. Some models have done binary classification, others have implemented biased algorithms based on the origination source, others have created knowledge graphs for path traversal and ranking to determine the likelihood of truthfulness. The approach taken here is still up for debate, but will be informed by the literature review and experimentation.

Models used in the literature include ML algorithms in conjunction with NLP techniques such as SVM, KNN, random forest, gradient boosting, and Naive Bayes. Other references have fine tuned LLMs on FEVER and other datasets, such as BERT, neural networks (FNN, RNN, and GRU). This project will likely include a handful of these models throughout the process specialized for each subset process which follows the general architectures in the literature.

The following references were used. Full citation of the papers will be implemented in upcoming journals to be incorporated in future documentation formally.

1. Truth is Universal: Robust Detection of Lies in LLMs
2. A Survey on Automated Fact-Checking
3. FactCheck Editor: Multilingual Text Editor with End-to-End fact-checking
4. ClaimBuster: The First-ever End-to-end Fact-checking System
5. Building a Better Lie Detector with BERT: The Difference Between Truth and Lies
6. Automated Fact Checking: Task formulations, methods and future directions
7. Towards automated check-worthy sentence detection using Gated Recurrent Unit

Methodology

Overview

The approach to this project will be a combination to an experimental and data-driven approach. The fundamental goal is to create a system that can take input, convert it to text, detect the claims to fact check, retrieve evidence, and determine the truthfulness. Reporting the evidence as well as the statistical likelihood based on the sources, the similarity and overlap, are the estimated confidence level in the classification of true, or not true. The data-driven component will come when these features developed are turned on and off, embeddings for evidence retrieval, RAG techniques, and LLM fine tuning methods are implemented to see their effect and influence on the correctness of the output.

Models and Algorithms

The project will consist of many ML and NLP algorithms throughout, some of which are not entirely obvious yet. Loosely, the preprocessing step will include a heavy amount of regexp, text manipulation and normalization such that statements can be classified. This can be as complex as using neural networks as done in the literature, or as simple as SVM, which also does pretty well. SVM will likely be used initially. After text classification and preprocessing, forming search content for information retrieval will include formatting triplets via NLTK tagging and SentenceTransformer parsing to break down statements into queryable formats. These queries will be used to search the databases for similarity matches, and if possible, contradiction matches via logical analysis of the context and text. This part of the project is slightly ambiguous still but is being fine tuned currently. After information retrieval, the evidence is classified via random forest, gradient boosting, or simple Naive Bayes, to determine the credibility of the N nearest documents. This is what generally informs the statistical outcome of the results. Further into the project the order of operations could change. Leveraging fine tuned language models like BERT, the truth detection could be incorporated into the LLM output. The accuracy of this model can be compared via the process described above and iterated until either the correct evidence is found (for training). This can be compared analytically to the output when prompted by one of the AI models (Claude, GPT, Gemini) with and without search being active.

Tech Stack

- Backend: Python, FastAPI, Uvicorn
  - Python and FastAPI for common backend setup that easily configures with open source libraries
- Frontend: JavaScript, React, Vite
  - JS, React/Vite for common frontend setup that has significant documentation (this is a new skill/toolkit)
- VCS/Deployment: Git, GitHub Actions for CI/CD, AWS
  - Git is universal and CI/CD necessary for ensuring app continuity
- Database: PostgreSQL, Pinecone
  - RDBMS systems from 550 work, but unsure whether MongoDB, or something RAG suitable like the common Pinecone vector database is the right choice. For now, working with the basic PostgreSQL for User storage and leveraging Pinecone for initial RAG implementation.
- NLP: SpaCy, NLTK, HuggingFace Transformers
  - Literature uses these libraries which drives a lot of the configuration and environment setup.
- Containerization: Docker (Docker compose)
  - Likely overkill for this project, but wanted experience incorporating Docker compose for frontend, backend server hosting locally for testing as well as containerization for hosting a live website that can scale with AWS
- EDA Packages: Jupyter Notebooks, Pandas, Matplotlib, Seaborn, NLTK, SpaCy, Transformers, Torch
  - Similar response to the NLP libraries. These will produce the trained models and processes that will be exported to the backend to be used as inference models rather than have these bulky libraries hosted and running real time.

Implementation Strategy

Implementation has been parallelized as much as possible to ensure a minimum viable product is always as close to being delivered as possible. The backend will rely heavily on open source libraries as that is most of the industry standard (based on the literature). Where relevant and logical, supporting functions will of course be written. The frontend is a simple implementation with much of the development being a learning process, building off the content from 550. A notebook is used to build out the backend preprocessing that is then moved into OOP based structure for the main functionality. Sentence analysis, classification, and all other NLP tasks are built out with test cases and tested against the common dataset from literature (LWIC, FEVER, etc.)

Evaluation Metrics

This project will rely heavily on mostly the classic NLP metrics: accuracy, precision, recall and F1. These metrics are critical because different error types have different implications such as high precision ensures that when the system flags misinformation, it is likely correct, maintaining credibility, however, with high recall the system catches most false claims, fulfilling its core mission. But the balance between these two is imperative so F1 is necessary to capture the overall state.

When incorporating RAG, there is another metric that is likely a balance between precision for determining the correct, or relevant documents (precision@k in the literature) and the overall correct evidence within the passage which relates more closely to the F1-score mentioned earlier.

Datasets and Models

Datasets

All data being used is open source. FEVER, Factily, Wikidata, and any other data sources are (currently) being considered as "true" - verified and trusted sources.

- **FEVER** – A large-scale dataset for fact-checking based on Wikipedia.
  https://fever.ai
- **Factify** – A dataset and benchmark for factuality detection.
  https://github.com/surya1701/Factify-2.0
- **Wikidata** – A collaboratively edited knowledge base from the Wikimedia Foundation.
  https://www.wikidata.org
- **ClaimBuster** – A dataset for factual text claims. 10.5281/zenodo.3836810

- **BERT** – A language model developed by Google AI.
  https://arxiv.org/abs/1810.04805
- **T5** – A unified framework that casts all NLP tasks as text generation problems.
  https://arxiv.org/abs/1910.10683

Models

The size of the datasets used vary. FEVER is about 125K rows of classified textual claims from Wikipedia. ClaimBuster contains approximately 32K rows of facts taken from debates that have been manually checked. Much of the preprocessing stage has been vetted due to literature overlap with the common datasets, however, that process is currently being implemented for this project. The datasets being used will be split for dev/train/test with cross fold due to the relatively limited data available, or more datasets from the literature will be pulled in as needed. The FEVER dataset will follow the subject-object-predicate triple format as mentioned earlier, however, that is not entirely uniform with other datasets, so considerations will need to be made for that going forward.

There are no ethical, or legal, considerations for the data collections methods. There is certainly the factor of bias inclusion given these datasets are majority restricted to English and US scopes, however, that is known. Future work related to this should extrapolate beyond that limitation.

Data augmentation is likely not necessary for this project. If more data is needed, pulling transcripts from audio files could inflate the pure text limitation, however, synthetic factual data is likely beyond the scope of the NLP limitations.

At this point, the hyper-parameters needed to be tuned are unknown. When considering fine tuning BERT for later implementation that will be a learning process that I have not entirely scoped out, however, for learning random forest, SVM, Naive Bayes, or other simpler ML classifiers the parameters will be learned via iteration or grid search for optimization.

Potential problems I foresee are related to scoring and evaluation. Overfitting is less of a risk here as the number of features is quite limited, as well as the available data. Under-fitting with unintentional biases is much more likely. This is mitigated by tracking performance metrics, but also correctly applying the folding methods for for dev/test/train data splits. When considering the performance metrics on the NLP results, determining the accuracy of the fact (true, not true) may be valid in an absolute sense, however, if the evidence used is incorrect that implies a false positive. Classifying and rectifying these failures will prove to be nuanced and likely the biggest hurdle within this project. Mitigation is likely going to include significant development in the preprocessing stage and text analysis prior to the information retrieval stage.

Project Plan and Timeline

Week 4: Create Initial Backend Model

- Implement sentence embedding baseline using SentenceTransformers
- Build similarity-based classification
- Integrate Wikipedia API for evidence retrieval
- Create model evaluation framework
- **Deliverables:**
  - ☐ Baseline fact-checker achieving on FEVER validation set
  - ☐ Wikipedia API integration for real-time evidence retrieval
  - ☐ Evaluation metrics (accuracy, F1, precision/recall)
  - ☐ Create evidence ranking and filtering system

Week 5: Build API Endpoints

- Build hooks for external API calls in backend
- Create skeleton UX/UI features for displaying backend results
- **Deliverables:**
  - Skeleton app and API endpoints in place.

Week 6: Develop Backend Model Further

- Implement advanced models (e.g., T5, BERT) via fine tuning on relevant datasets

- Experiment with RAG techniques for evidence retrieval (Pinecone)

Week 7: Determine and Test Performance Metrics

- Sentence classification performance (F1).
- Document embedding and retrieval performance (MTEB)
- Truth detection performance (accuracy, precision/recall).

Week 8: Iterate on Backend Model

- Implement sentence embedding baseline using SentenceTransformers
- Build similarity-based classification
- Integrate Wikipedia API for evidence retrieval
- Create model evaluation framework

Week 9: Create User Database

- Implement a PostgreSQL database for user management (AWS RDS).
- Record user inputs and results for future analysis.

Week 10: Full Implementation and Testing of App

- Finish API hooks and UX/UI features.
- Conduct end-to-end testing of the application.
- Create backlog of features to fix and improve.

Week 11: Conduct Initial "Search" Testing

- Test output with and without truth detection to determine influence on LLM.

Week 12: Documentation and Final Report

- Gather user documentation and create final report.
- Prepare presentation materials.
- Update Github README and documentation for public release.